



Iterative Infrastructure with ZFS and Zelta

Daniel J. Bell
BSDCan 2026

Six years. Hundreds of millions of snapshots. Zero package dependencies.

[Pause]

Right now, we're all feeling the pain on the hardware side. Who the heck is eating all of our drives? And what are we going to do about it?

A huge chunk of the forward drive inventory is going to AI. Frontier open-weight models are already 600–800 GB.

The Drive Math

600–800 GB

One frontier open-weight model

× 3 variants = 1.8–2.4 TB of conventional storage

× 3 variants = 1 model + deltas with ZFS clones



Iterative Infrastructure with ZFS and Zelta

2

So here's a little drive math.

[Click 1:]

Three fine-tuned variants of the same base model used to mean three full copies. With conventional workflows that's triple the storage cost. Or...

[Click 2:]

it's one model plus deltas. That's the world ZFS and Zelta are built for. Let's get the most out of our deltas.

25 Years Looking for the Perfect Backup

About Me

- Sysadmin and fractional CTO
- Financial services, healthcare
- Founded Bell Tower
- Author of two FreeBSD ports: zelta & ngbuddy

Daniel J. Bell
<https://zelta.space>



Iterative Infrastructure with ZFS and Zelta

3

I'm Daniel Bell.

Here are a few things about me, but most importantly I've been rocking BSDs since the 90s.

I've also been doing infrastructure in regulated environments for a long time; the kind of places where a bad backup isn't just data loss, it's a regulatory event.

I wrote Zelta because I needed a replication tool for my FreeBSD Private Cloud that matched how I think about backups and iteration. What started as a backup tool turned into an infrastructure and safety philosophy. As an added benefit, it helps me squeeze a lot of space out of every drive, and that's what this talk is about.

The Missing ZFS Building Blocks

What We Have

- Powerful, but verbose
- Scriptable, but ad-hoc
- Reachable, but blind
- Scheduled, but manual

What We Need

- Simplified output
- Orchestrated workflows
- Remote awareness
- Intelligent policies



Iterative Infrastructure with ZFS and Zelta

4

Zelta exists because we have a powerful filesystem with amazing primitives that are hard to stretch as far as they can go, but I don't need something that replaces the built-in ZFS utilities.

What's missing is **composition**; clear output that makes sense at 3 a.m., tools that understand the relationships of replicas, workflows that handle edge cases without blowing up, and policies that make sense.

Zelta: Open, Portable, and Opinionated

Standards

- BSD 2-Clause License
- Ultra-portable
- Unix principles
- Point-of-use documentation

Defaults

- As safe as possible
 - Clone/Rename vs. overwrites
- Metadata aware
 - No forced structure or naming
- Remote and recursive



Iterative Infrastructure with ZFS and Zelta

5

I built Zelta with some specific principles in mind.

- It has to be permissively licensed, because I want to collaborate with people freely and want everyone to have zero friction using it anywhere.
- It has to be ultra-portable, so I can save a decrepit TrueNAS server with nothing but a rescue drive and Kernighan awk.
- I need Unix principles all the way through, so everything composes together fluidly with other tools, and scales.
- And I want point-of-use documentation, because that's where documentation actually belongs.

[CLICK]

ZFS comes with enough foot-guns, so I want Zelta to be as safe as possible. I never want to be pressured to delete something that could be important in a recovery scenario.

I want flexibility, so no forced naming conventions and no required dataset structure on any system.

Zelta's safety and UNIX-philosophy extends well to many projects and environments. In particular, I'm very proud that we've been adopted by the Sylve project, an amazing VM, jail, and storage control plane for FreeBSD. We don't have the most GitHub stars, but we have big fans in high places.

Remote, Recursive Operations for ZFS

zelta



Iterative Infrastructure with ZFS and Zelta

6

Zelta's job is to extend ZFS operations to be remote, recursive, safe, and auditable.

[Click 1 — show zelta verbs]

Here are Zelta's verbs; you can just run `zelta usage` or `zelta help` to learn about them. Each simplifies one or more ZFS commands.

[Click 2 — show zfs mappings]

For instance, `zelta match` takes five billion lines of `zfs list` output and tells you, in just a few lines, what you actually need to know: *can I replicate this, or are we cooked?*

Zelta Backup does ZFS Send and Receive, but with divergence handling, remote awareness, and mountpoint safety. Zelta takes complex, error-prone ZFS workflows and handles the scary edge cases for you.

Remote, Recursive Operations for ZFS

zelta	{	match	= zfs list
		backup	+= zfs send/recv
		clone	+= zfs clone
		failover	+= zfs set
		rotate	+= zfs rename
		rebase	+= zfs diff
		policy	+= xargs
zprune		= zelta prune + zfs destroy	



Iterative Infrastructure with ZFS and Zelta

7

And **zprune** is a new and deliberately separate command, because it destroys snapshots.

I wanted Zelta itself to *never* be destructive. Even by accident. If someone runs a Zelta command and something goes wrong, I never want them to be able to say “Zelta ate my homework.” So I kept the explosive bits in their own script.

It also turned out to be a useful separation of concerns. **zelta prune** can safely pipe retention output to other systems with minimal privileges, while zprune handles summaries and nuking.

Zelta 1.2 Backup Defaults in Action

What you want to do	zelta backup Options
Recompress/Re-encrypt	<code>--large-blocks</code>
Snapshot with a prefix	<code>--snap-prefix=<i>name</i></code>
Limit based on pattern	<code>--include=<i>glob</i>, --exclude=<i>glob</i></code>
Bookmark latest sent	<code>--bookmark</code>
Resume, Recursive, Raw	<code>[default]</code>
Intermediate	<code>[default; stepwise with pattern]</code>

```
zelta backup --snap-prefix=daily --include='@daily*' \
source:sink/ds tank:target/ds
```



Iterative Infrastructure with ZFS and Zelta

8

Zelta 1.2, BSDCan edition, is available to preview its intuitive new features and wonderfully boring defaults.

When possible, Zelta options pass through to the ZFS flags they influence. For example, ZFS Send flags will override Zelta's defaults and enforce those flags for `zfs send` commands it runs.

Three filters—depth, exclude, and include—now work globally across Zelta and are more flexible. For example, you can filter snapshot or dataset patterns, which tells Zelta Backup to limit replication to only snapshots that match.

We added a bookmark option that tags the hostname of the recipient host, providing a bit of handy point-of-use telemetry.

There are a few other new features I'll mention during the talk, but most importantly: the majority of Zelta use cases need just your one or two dataset endpoints and **no switches at all**.

One Mental Model, Every Command

1 Most Specific Option Wins

- Default
- Config
- Environment
- Argument

2 Intuitive Defaults

- Global flags:
 - --dryrun
 - --verbose
- Common Standards

3 Visible Safety Boundary Visible

- zelta: Nondestructive
- zprune: Destroy snaps
- zfs: Anything goes!

Endpoint Format: `user@host:pool/ds@snap`



Iterative Infrastructure with ZFS and Zelta

9

Six years of operator feedback also taught me what makes a tool both useful and pleasant to use in production.

First, predictable hierarchical option behavior. Basic UNIX stuff, and the other options follow that lead.

[CLICK]

We have expressive dryrun and verbose modes for every option that's proven to be a great ZFS training tool at my company. For endpoint resolution, we use a simple SCP-style syntax, so it feels familiar.

[CLICK]

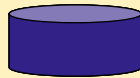
Zelta isn't designed to replace ZFS for all operations, but instead to keep the safety boundary visible. If the command starts with the word "zelta," nothing is destroyed, and zprune can't delete datasets or clone origins.

These safety boundaries really matter in production. Last year, a third-party

update nuked a client's domain controller in the middle of the night. My junior sysadmin took the call, and this was the first time `zelta revert` was used in production.

Not only did she get the VM back online, but using Zelta instead of `zfs rollback` meant we retained the broken version of the VM and were able to get to the bottom of the crash.

Start Here: Zelta Backup



```
zelta backup alpha:ark/ds beta:box/ds
```



Iterative Infrastructure with ZFS and Zelta

10

I'd like to show some fancier Zelta patterns, but let's start with some basic backups.

[Click:]

Zelta Backup, followed by a pair of dataset endpoints replicates the first to the second, along with all of its descendants.

Here, we have a production dataset on host Alpha backing up to a backup server named Vault.

[Click:]

Zelta Backup Alpha, Beta. Let's call the pair of hosts twins. They have similar hardware, so either one can be the active host of our datasets.

[Click:]

Just repeat the commands on a schedule to keep them up to date. You can repeat them safely no matter what state the systems are in. If something goes wrong, Zelta will tell you.

If nothing has changed on the source, Zelta won't create a pointless new snapshot. It just confirms that your replicas are already up to date.

Start Here: Zelta Backup

Sync deltas regularly:

```
% zelta backup alpha:ark/ds beta:box/ds
% zelta backup --snap-prefix=daily \
    --include='@daily*' alpha:ark/ds vault:vat/ds
```

Swap production direction:

```
% zelta failover alpha:ark/ds beta:box/ds
```



Iterative Infrastructure with ZFS and Zelta

11

Here are the commands we just used.

For regular syncing, **zelta backup** replicates the full history from the source to the target.

We can also get more selective. In the second example, we sync Alpha to Vault with the snapshot prefix “daily,” and filter it to send only those snapshots. This can be handy if you want to run backups with different intervals and naming schemes.

[Click]

A replica dataset can only receive new snapshots if it isn't modified; this goes for both backups and failovers. If you want to move a dataset, use **zelta failover** to perform a final sync and swap the read-write properties of our twins.

Next, we can talk a bit about dataset clones, and how they can help when something goes awry.

Zero-Cost References For ZFS

Dataset Clones

A writable dataset whose initial state is shared with a snapshot.

- Zero bytes until divergence
- Travels across pool and host boundaries

Remote Cloning Example

Clone via replication:

```
% zfs send -i apool/origin@snap \
  apool/ds@snap | zfs receive \
  -o origin=bpool/origin@snap bpool/ds
```

Zelta Recursive Equivalent:

```
% zelta backup --origin bpool/origin \
  apool/ds bpool/ds
```



Iterative Infrastructure with ZFS and Zelta

12

Most people who hear the word 'clone' just think about a regular 'copy', but a ZFS clone consumes no space.

One of Zelta's goals is to make using these zero-cost references more accessible. For example, Zelta provides safe alternatives to several destructive ZFS actions using dataset clones.

In addition to being free, clones are portable and can replicate to a different pool or host. You can sync one as long as its origin, the snapshot it's referenced from, is available on both sides.

[CLICK]

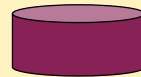
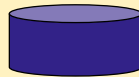
Replicating clones, especially recursive ones to a remote server, can be tricky with ZFS commands. Zelta provides several different options to help, such as Zelta Backup Origin.

The practical result is that you can spin up as many independent, writable versions of a dataset as you want on as many servers as you want, and they only cost you storage for the parts that actually change.

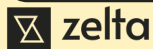
Clones for Recovery: Zelta Revert

Alpha:ark

Beta:bin



```
zelta revert alpha:ark/ds1
```



Iterative Infrastructure with ZFS and Zelta

13

Let's say we need to return a dataset to the state of a previous snapshot; a common solution is the destructive ZFS Rollback, which destroys everything to the previous point.

A lot of ZFS pros avoid this: they rename the dataset to keep the changes safe and clone it to restore its old state.

You can do this rename and clone tango in one step, recursively, with Zelta Revert.

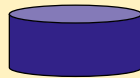
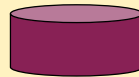
[CLICK]

We've rewound our dataset but now the target has diverged from the source, which blocks usual replication. Let's fix it.

Clones for Recovery: Zelta Rotate

Alpha:ark

Beta:bin



```
zelta rotate alpha:ark/ds1 beta:bin/ds2
```



Iterative Infrastructure with ZFS and Zelta

14

Use Zelta Rotate to rename our target and create a clone in its place based on a source delta.

[CLICK]

The source and target now have a matching snapshot, and replication can continue normally. Zelta Rotate helps other continuity problems as well.

Clones for Recovery: Zelta Rotate

Rename & clone replicas to restore sync continuity:

- If the source was cloned (zelta revert)
- If the source was rolled back (zfs rollback)
- If the target has diverged

```
% zelta rotate alpha:ark/ds beta:box/ds
```



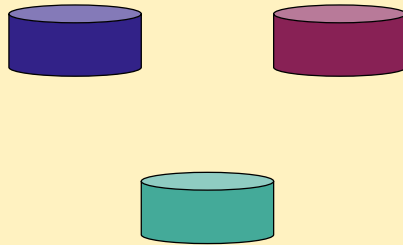
Iterative Infrastructure with ZFS and Zelta

15

Zelta Rotate fixes replica divergence in three scenarios: if the source was cloned, like our Zelta Revert example, if the source was rolled back, or if the target has diverged.

It's also handy for iterative QA workflows, which we'll discuss shortly.

Zelta Twins: A Warm Failover Pair



```
zelta backup alpha:ark/ds beta:box/ds # No op  
zelta backup alpha:ark/ds vault:vat/ds # No op
```



Iterative Infrastructure with ZFS and Zelta

16

Using everything we know so far, we can make a warm failover pair using Zelta Backup commands.

[CLICK]

When data changes on Alpha's dataset, Zelta Backup from Alpha to Beta will sync.

[CLICK]

But Beta to Alpha won't, because Beta is set read-only and can't change. Zelta won't take a snapshot unless there's something new. That's important for keeping sync continuity in the right direction.

[CLICK]

Now, we're using Zelta Failover to promote Beta to the read-write copy.

[CLICK]

Now our sync direction has reversed. Zelta will no longer snapshot or sync Alpha to Beta.

[CLICK]

ZFS really does make this beautifully simple.

Let's make things a bit more complicated.

Distributed Storage Is *Okay*

Distributed Wins

- Node resilience
- Massive scale
- Geographic distribution

ZFS Wins

- Integrity
- Recovery
- Performance
- Flexibility/tunability
- Portability
- Efficiency



Iterative Infrastructure with ZFS and Zelta

17

[If no one laughs:] Distributed Storage is Ok!

The title is facetious: Of course, distributed storage is amazing: they can scale massively and keep things running when nodes die. Ceph is the best choice for many workloads.

[CLICK]

But a lot of the storage growth we're seeing right now, like VMs, databases, and inference, have different needs. These workloads need to be performant, correct, durable, well-tuned, and recoverable.

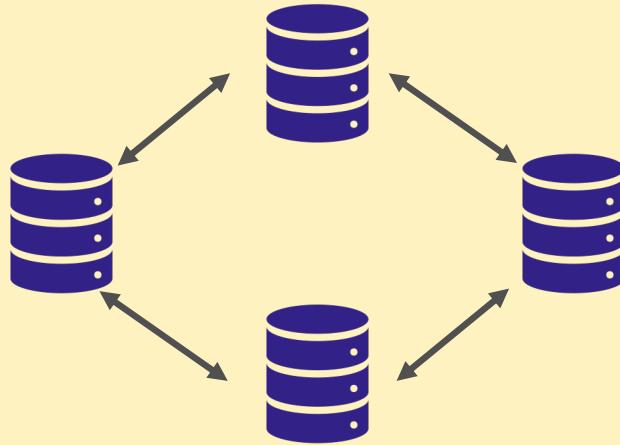
And a distributed system means that when something goes wrong, it goes wrong at scale. If ransomware encrypts your files, the cluster does exactly what it was built to do. It dutifully replicates the encrypted blocks everywhere. Don't forget about the robot army of vite-coded script kiddies getting smarter by the day. And they never sleep.

ZFS doesn't make you immune. But I've recovered from catastrophes on ZFS that

simply could not be resolved as efficiently or painlessly in most other environments.

ZFS is actually built for performance and durability where most of our workloads live.

Zelta Policy: More Replicas



Iterative Infrastructure with ZFS and Zelta

18

Let's talk about managing lots of twins with bidirectional failovers, disaster recovery sites, ephemeral containers, and read-only replicas distributed where you need them.

It can be a lot to manage, especially when your different workloads need different ZFS options.

This is where organizing our backup jobs with Zelta Policy can help.

Zelta Policy: More Replicas

```
JSON: 1
JOBS: 8
DAL1:
  host00.bts:
    options:
      import: targets/vault1.den0.yaml
    datasets:
      import: sources/host00.bts.yaml
```



Iterative Infrastructure with ZFS and Zelta

19

Here's a snippet of an example Zelta Policy file.

My company, Bell Tower, runs our policies from several hardened OpenBSD Zelta Bastions to orchestrate thousands of datasets across many sites. Having an orchestrator that isn't a storage node is critical for security and operational clarity at scale.

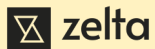
In our configuration, we can define any Zelta command line option at any point in the hierarchy; global definitions, by user defined Site name—DAL1 in our example—or by host. “Site” is the unit of concurrence; Zelta Policy will orchestrate one site per number of JOBS defined. If you have different patterns for multiple hosts, you can define them repeatedly and Zelta will do the right thing.

Most log messages emit directly from Zelta Backup, which supports human readable logs or JSON.

Zelta Policy: More Replicas

```
% cat targets/vault1.den0.yaml  
ADD_HOST_PREFIX: 1  
BACKUP_ROOT: vault1.den0:data1/Backups
```

```
% cat sources/host00.bts.yaml  
- ssd00/jail/app_abc  
- ssd00/jail/app-dev_abc  
- ssd00/jail/db_abc  
- ssd00/jail/gitea_abc  
- ssd00/vm/pbx.abc
```



We keep our policies light and manageable by keeping source datasets, backup targets, and option lists in separate files. Here are some options describing how to name the snapshots on the receiving side and source dataset definitions. It's very flexible.

Zelta Policy: More Replicas

```
% zelta policy --dryrun host00.bts
SOURCE                                TARGET
host00.bts:ssd00/jail/app_cloud0     vault1.den0:data1/Backups/app_cloud0
host00.bts:ssd00/jail/app-dev_cloud0 vault1.den0:data1/Backups/app-dev_cloud0
host00.bts:ssd00/jail/db_enterprise  vault1.den0:data1/Backups/db_enterprise
...
% zelta policy --dryrun --verbose host00.bts
+ ZELTA_EXCLUDE="/backups,/swap,/tmp" ZELTA_SNAP_TIME="8h" ZELTA_SEND_INTR="0" zelta
backup 'host00.bts:ssd00/jail/app_cloud0' 'vault1.den0:data1/Backups/app_cloud0'
+ ZELTA_EXCLUDE="/backups,/swap,/tmp" ZELTA_SNAP_TIME="8h" ZELTA_SEND_INTR="0" zelta
backup 'host00.bts:ssd00/jail/app-dev_cloud0' 'vault1.den0:data1/Backups/app-
dev_cloud0'
+ ZELTA_EXCLUDE="/backups,/swap,/tmp" ZELTA_SNAP_TIME="8h" ZELTA_SEND_INTR="0" zelta
backup 'host00.bts:ssd00/jail/db_enterprise'
'vault1.den0:data1/Backups/db_enterprise'
```



This example Zelta Policy dryrun output might not seem pretty from this far away, but I promise it is absolutely stunning when I'm troubleshooting.

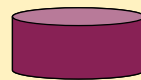
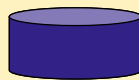
Dryruns shows a tidy pair of filtered source and target pairs, while a Verbose Dryrun provides the gory details of exactly what Zelta Policy will run.

Zelta Policy runs everything defined in its policy file by default, but can be filtered by just adding a Site, Host, dataset, or dataset leaf name.

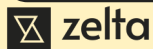
Iterative Infrastructure

Alpha:ark

Beta:bin



```
zelta clone alpha:ark/ds1 alpha:ark/ds2 \  
          beta:bin/ds1 beta:bin/ds2
```



Iterative Infrastructure with ZFS and Zelta

22

When we're dealing with this much data, we'd better make the space count! Let's get back to two more cloning patterns I use.

Zelta Clone with two operands recursively clones a dataset. Cloning is a pool action, so we can only clone it locally.

[Click] [Wait]

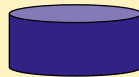
[Click]

But Zelta Clone with four operands also fires a Zelta Rotate, retaining sync continuity for itself and its clone. Both pairs of datasets, DS1 and DS2, can now function as independent units, but consume no space on either pool until they're modified.

Iterative Infrastructure

Alpha:ark

Beta:bin



```
zelta backup --target-origin beta:bin/ds1 \  
alpha:ark/ds2 beta:bin/ds2
```



Iterative Infrastructure with ZFS and Zelta

23

If you already made the clone and want to sync it with its zero-cost clone reference, we can do that as well.

[CLICK]

You can provide Zelta Backup with a Target Origin to replicate the clone reference on the target side. That first backup is free.

Iterative Infrastructure

Local + Remote Clone:

```
% zelta clone alpha:ark/ds1 alpha:ark/ds2 \  
beta:bin/ds1 beta:bin/ds2
```

Local Clone:

```
% zelta clone alpha:ark/ds1 alpha:ark/ds2
```

Sync a Clone With a Reference:

```
% zelta backup --target-origin beta:bin/ds1 \  
alpha:ark/ds2 beta:bin/ds2
```



Iterative Infrastructure with ZFS and Zelta

24

You can accomplish amazing things with ZFS workflows based on these.

Three years ago, I was working for a financial firm who ran a full web stack I migrated to ZFS, bhyve, and jails. A database, web front ends, multiple backends, and most disagreeably, four Windows boxes. All of it was running in parallel as a single application stack.

They needed to test daily versions against **real** production data: multiple environments running at the same time for different branches, every day. This is where ZFS really shines.

The daily deltas for the QA versions were around 1% of its production total, and we got hot, independent, testable environments from a single base. Amazing.

Zero-Cost References For ZFS

Clones

A writable dataset whose initial state is shared with a snapshot.

- Zero bytes until divergence
- Travels across pool and host boundaries

Block Cloning

Copied files reference existing blocks instead of writing new ones.

- Source-side efficiency
- Copying takes no space
- Really!

Fast Dedup

Identical blocks are stored once.

- Includes data from incoming replicas
- Great for backup pools



Iterative Infrastructure with ZFS and Zelta

25

Recently, I've been experimenting with workflows using the two other types of iterative storage for ZFS.

An exciting somewhat recent addition to ZFS is Block Cloning.

When you copy files or parts of files, ZFS can reference the existing blocks on disk instead of writing new ones. As with dataset clones, you only pay for the new data. Block cloning happens automatically and works between filesystem boundaries on the same pool. The fact that copying files no longer consumes space feels like magic.

On its own, block clones provide only source-side efficiency. New block clone references must be replicated again when using ZFS Send operations.

CLICK Fast Dedup block

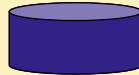
There's also the new Fast Dedup feature, replacing ZFS's old, painful, slow, and memory hungry deduplication system. The new version is... painful, slow, and

memory hungry, but significantly better.

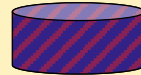
But if you have a case where it works, it's excellent. Identical blocks that arrive from different sources only cost space once. That means you could potentially get the space savings of **block clones** on both sides.

Zelta Rebase (*Preview Feature*)

Alpha:ark



Beta:bin



```
zelta rebase alpha:ark/ds1 beta:bin/ds2
```



Iterative Infrastructure with ZFS and Zelta

26

Here's an experimental workflow our team is testing that ties together everything we've discussed.

Imagine we have a fleet of FreeBSD jail replicas. They're all regular, "thick" jails—they're lived in and have changes, their own databases, and unique data. But we want updates from their origin.

With dataset clones and block cloning, we can do exactly that.

First, we perform a Zelta Rebase. [CLICK]

Now, we'll copy a subset of files back, based on a mapping file included with the update from the base. [CLICK]

That's it. We have a jail patched from upstream that retained some of its old data, and nothing was lost in the process.

Another way a friend and colleague said it, "Thick container convenience with thin container efficiency."

This feature hasn't even started QA yet, but it's actually just a composition of Zelta Rotate and copy. The hard part is figuring out the ZFS Diff based map file.

zelta prune

```
zelta prune source:sink/ds target:tank/ds
```

- First operand, **source**, is acted upon.
- Optional **target** is for “prune guard” comparison.

Same options, two commands:

- **zelta prune**: Provides pipeable output and previews
- **zprune**: Provides summary, destroys with confirmation or **-f**

zelta prune --visual:

-  = protected snapshot
-  = candidate for destruction
- Oldest top left, newest bottom right



Iterative Infrastructure with ZFS and Zelta

27

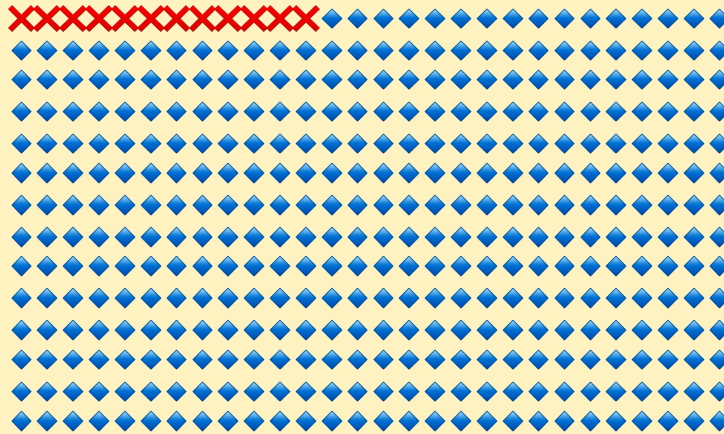
Of course, the most surefire way to save space is by deleting stuff you don't need.

zelta prune helps you plan snapshot retention policies.

It provides previews and pipeable output and is the engine for **zprune**, which actually destroys snapshots.

The visuals you're about to see are real output. Blue diamonds are protected. Red Xs are the candidates for deletion.

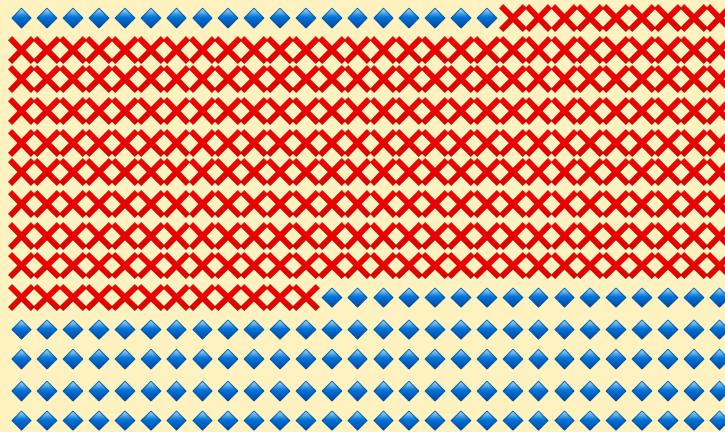
```
zelta prune --prune-size=150g -depth=1 sink/ds
```



Why do we murder our snapshots? Usually because we're low on space.

The option "prune size" selects the oldest snapshots until we've freed the desired amount. 150 gigabytes of red Xs of doom.

```
zelta prune --prune-size=150g \  
--prune-guard=unsynced sink/ds target:tank/ds
```

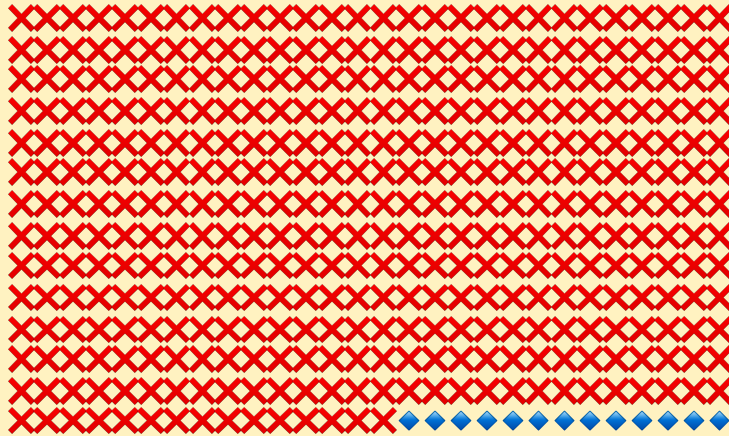


But what if we need to make sure we have copies of everything? Those old snapshots might be the only copy.

Add “prune guard unsynced” and Zelta protects anything that the target, the second operand, hasn’t seen. The diamonds on the top row tell us the vault is missing history.

It looks like our backup target never received a full sync.

```
zelta prune --prune-time=30days sink/ds
```



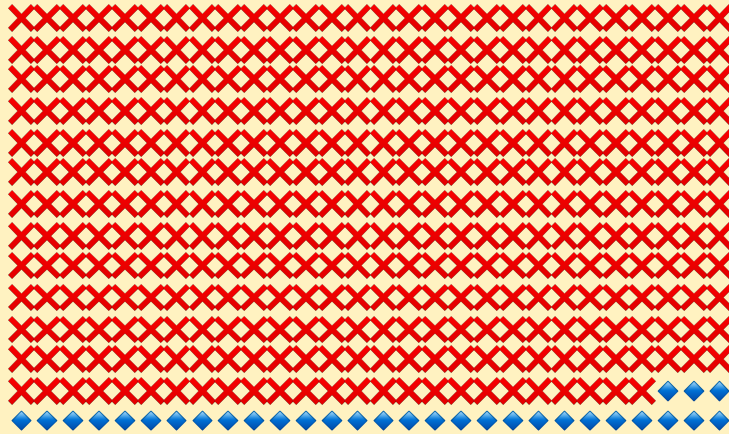
Iterative Infrastructure with ZFS and Zelta

30

You can also prune by time or by count from right now. “Prune Time” and “Prune Num.”

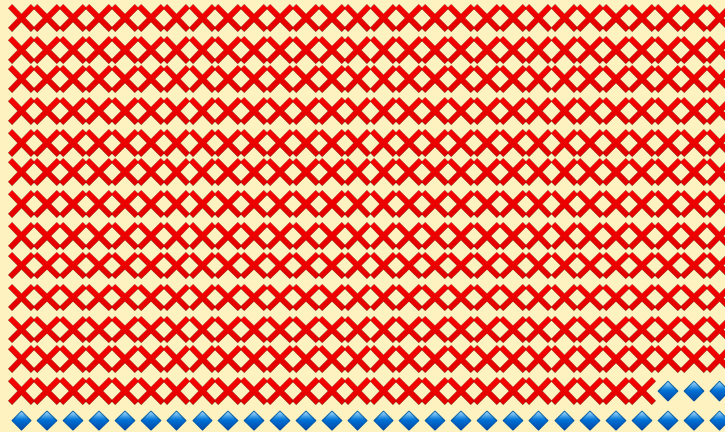
“Prune Time 30 days” keeps 30 days... [CLICK]

```
zelta prune --prune-num=30 sink/ds
```



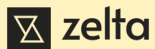
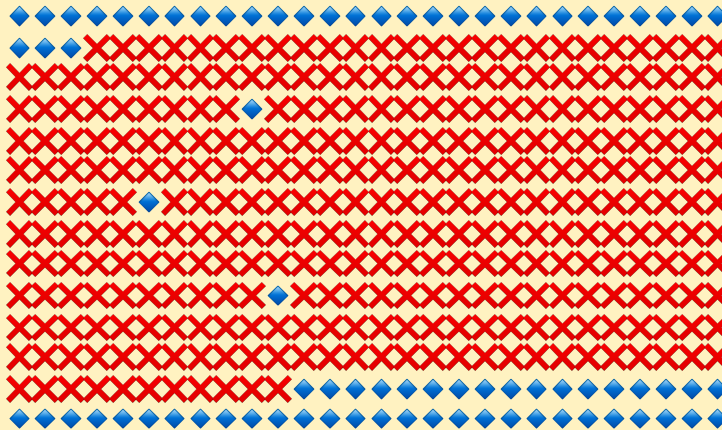
or keep the most recent 30 snapshots with “prune num 30”. [CLICK]

```
zelta prune sink/ds
```



The default policy keeps whichever is more restrictive: both 30 days *and* at least 30 snapshots, so in this example, the default options are identical to "prune num 30."

```
zelta prune --include='@zelta*' sink/ds
```

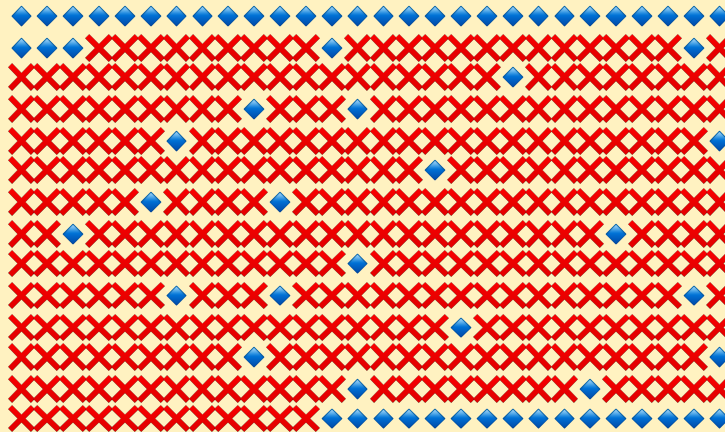


For more advanced retention policies and to play nice with other retention tools, you can use Zelta's universal filtering options `depth`, `exclude`, and `include` to narrow the set.

Here, we're considering only snapshots that start with the word Zelta, while leaving on the default 30 days and 30 snapshots.

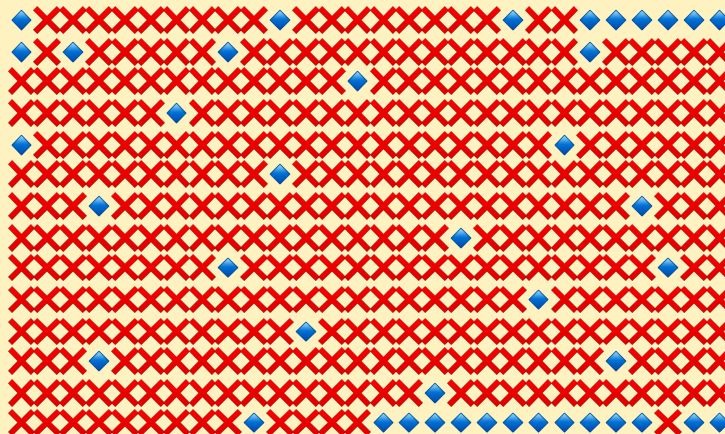
Some older snapshots tagged "Zelta" will get cleared out while a few with different names stay. In our example, the first X must be the point where we started tagging the snapshots with the word "zelta."

```
zelta prune --include='@zelta*' \  
--prune-grid='1week' sink/ds
```



For aging data lifecycle policies, we have a “prune grid” that works similarly to zrepl’s great retention system. We can define simple options, like this one that keeps one snapshot per week forever.

```
zelta prune --prune-grid='30x1day,52x1week,1year' sink/ds
```



or a complete list of data lifecycle windows.

So you can make pretty much any retention policy with zelta prune.

All the options stack.

You can filter first by included and excluded dataset or snapshot names, dataset depth, and replication relationship.

Then select snapshots to destroy by size, time, count, or aging retention grid.

```
zprune --prune-size=150g sink/ds
```

```
zfs destroy sink/ds@2024010100%2026040105
```

```
12 snapshots (3% of 392) will be destroyed
```

```
179.7G total reclaimed (11% of 1.6T)
```

```
Destroy now? [y/N]
```



After you investigate your preferred command with **zelta prune**, execute your strategy with the destructive **zprune**.

What it lacks for in emojis, it makes up for by showing the exact **zfs destroy** commands it's about to run and a summary of how many snapshots will be smote and how much space you'll get back along with percentages for each dataset in your tree. Like all Zelta commands, they work recursively and remotely.

What's Next for Zelta

- Data-driven refactor, professional QA, and TDD
- New language for Zelta Bastion (not Zelta portable)
- Improve rebase, prune, and policy
- Stream mode for non-ZFS endpoints (S3, etc.)



Iterative Infrastructure with ZFS and Zelta

37

Years of working on my own with the ultraportable awk and shell limitations got Zelta pretty far, but it's time for the project to grow.

First, we're investing heavily in Zelta, and we will always maintain a highly portable and BSD licensed version. I'm never going to give up that flexibility for recovery scenarios. We've made some progress moving toward a professional data-driven refactor with test-driven development.

Since the beginning of the year, Zelta no longer needs to be installed on ZFS endpoints. We're running low on reasons to port every feature to 1977, so we've started to prototype more advanced workflows in Ruby. This gives us the agility to add additional security features, replication hints, ZFS Program workflows, and other things that are too much of a bear in Bourne and Kernighan Awk.

We'll also keep enhancing and adding features.

Questions so far?



Iterative Infrastructure with ZFS and Zelta

38

Now, I'd like to separate our concerns: Before we move onto talking about my approach to ZFS permissions and security model, any questions about ZFS iterative infrastructure concepts or Zelta?

Least Privilege, Always

Production Admin: Service management, non-destructive recovery

- *snapshot, rename, clone*

Failover Admin: Standby server replication & recovery

- *send:raw, snapshot, receive*

Snapshot Admin: Snapshot scheduling & retention policies

- *snapshot, destroy on production servers*

Backup Admin: Automated backup management, manual recovery

- *send:encrypted, receive from production to backup systems*

Retention Admin: Pruning and redaction on backup systems

- *hold, destroy on backup systems*



Iterative Infrastructure with ZFS and Zelta

39

I have the evidently rare belief that **sudo** is not a universal command prefix.

Segregation of duties isn't just a good idea - it's often legally required.

Here's my mental model of the 5 distinct and separate privilege sets. Obviously this doesn't refer to 5 different people or Zelta installations, though it could.

But we should set up our ZFS workflows so production systems can't touch backups. And of course, backups can't meddle with production.

[CLICK]

Here are the associated ZFS Allow permissions for the five security roles.

We're working on a zelta tool to help you detect and set appropriate permissions.

And while we're on the subject, my thanks to the prolific OpenZFS developer Rob Norris, who on my recent suggestion just opened an OpenZFS PR to extend zfs allow permissions with a new **send:encrypted** permission. That would limit the backup operator to only encrypted datasets, making it impossible for a backup operator to ever

see an unencrypted stream.

Coming soon.

OpenZFS Wish List

-  Userspace ZFS for untrusted, portable replication
- Block-clone-aware replication
- Zelta-ish dataset relationship tools in OpenZFS



Iterative Infrastructure with ZFS and Zelta

40

Onto my OpenZFS wish list. Well, my biggest wish is coming true. OpenZFS community members and funding from TrueNAS is making user space ZFS a reality. This opens the doors to new portability and security options. New ZFS on NetBSD, anyone? Exciting stuff.

I've been told my next wish list item is impossible, but I'll mention it anyway. Since deduplication is still a heavy lift, block-clone-aware replication would be so nice to have. I realize the block cloning just matured five minutes ago, but now I want it everywhere!

And finally, what if some of Zelta's features were part of the OpenZFS project itself? Some of the replica relationship stuff, like Zelta Match and easier clone tracking, would fit right into the ZFS command stack. Maybe?

Let's talk about it!

Let's Continue the Conversation!

Daniel J. Bell

zelta.space

belltower.it



Iterative Infrastructure with ZFS and Zelta

41

Thank you!